

Astron Finished Security + Attendance Files

Compatible with:

- Discord.js v14
- PostgreSQL
- Prisma
- Railway

1. Install Packages

```
npm install discord.js prisma @prisma/client dotenv ms
```

2. Prisma Schema

prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Attendance {
  id          Int      @id @default(autoincrement())
  guildId     String
  userId      String
  status      String
  createdAt   DateTime @default(now())
}

model Verification {
  id          Int      @id @default(autoincrement())
  guildId     String   @unique
  roleId      String
  channelId   String
  minAgeDays  Int      @default(7)
}
```

```
model AntiNuke {
  id          Int      @id @default(autoincrement())
  guildId     String   @unique
  enabled     Boolean  @default(true)
  logChannelId String?
```

Run:

```
npx prisma generate
npx prisma migrate dev --name init
```

3. Prisma Client

src/database/prisma.js

```
const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient();

module.exports = prisma;
```

4. Attendance Command

src/commands/attendance/create.js

```
const {
  SlashCommandBuilder,
  EmbedBuilder,
  ActionRowBuilder,
  ButtonBuilder,
  ButtonStyle
} = require('discord.js');

module.exports = {
  data: new SlashCommandBuilder()
    .setName('attendance-create')
    .setDescription('Create attendance panel'),

  async execute(interaction) {

    const embed = new EmbedBuilder()
```

```

        .setTitle('📅 Astron Attendance')
        .setDescription('Click below to mark attendance.')
        .setColor('#5865F2');

const row = new ActionRowBuilder().addComponents(
    new ButtonBuilder()
        .setCustomId('attendance_present')
        .setLabel('Present')
        .setStyle(ButtonStyle.Success),

    new ButtonBuilder()
        .setCustomId('attendance_late')
        .setLabel('Late')
        .setStyle(ButtonStyle.Secondary),

    new ButtonBuilder()
        .setCustomId('attendance_absent')
        .setLabel('Absent')
        .setStyle(ButtonStyle.Danger)
);

await interaction.channel.send({
    embeds: [embed],
    components: [row]
});

await interaction.reply({
    content: 'Attendance panel created.',
    ephemeral: true
});
}
};

```

5. Attendance Button Event

src/events/attendanceButtons.js

```

const prisma = require('../database/prisma');

module.exports = {
    name: 'interactionCreate',

    async execute(interaction) {

        if (!interaction.isButton()) return;

        if (!interaction.customId.startsWith('attendance_')) return;
    }
}

```

```

const status = interaction.customId.replace('attendance_', '');

const existing = await prisma.attendance.findFirst({
  where: {
    guildId: interaction.guild.id,
    userId: interaction.user.id
  }
});

if (existing) {
  return interaction.reply({
    content: 'You already marked attendance.',
    ephemeral: true
  });
}

await prisma.attendance.create({
  data: {
    guildId: interaction.guild.id,
    userId: interaction.user.id,
    status
  }
});

await interaction.reply({
  content: `Attendance marked as ${status}.`,
  ephemeral: true
});
}
};

```

6. Verification Setup Command

src/commands/verification/setup.js

```

const {
  SlashCommandBuilder,
  EmbedBuilder,
  ActionRowBuilder,
  ButtonBuilder,
  ButtonStyle
} = require('discord.js');

module.exports = {
  data: new SlashCommandBuilder()
    .setName('verify-setup')

```

```

        .setDescription('Setup verification panel'),

    async execute(interaction) {

        const embed = new EmbedBuilder()
            .setTitle('🔒 Verification')
            .setDescription('Click verify below to access the server.')
            .setColor('#2B2D31');

        const row = new ActionRowBuilder().addComponents(
            new ButtonBuilder()
                .setCustomId('verify_button')
                .setLabel('Verify')
                .setStyle(ButtonStyle.Success)
        );

        await interaction.channel.send({
            embeds: [embed],
            components: [row]
        });

        await interaction.reply({
            content: 'Verification panel created.',
            ephemeral: true
        });
    }
};

```

7. Verification Button Event

src/events/verifyButton.js

```

module.exports = {
    name: 'interactionCreate',

    async execute(interaction) {

        if (!interaction.isButton()) return;

        if (interaction.customId !== 'verify_button') return;

        const member = interaction.member;

        const accountAge = Math.floor(
            (Date.now() - member.user.createdTimestamp) /
            (1000 * 60 * 60 * 24)
        );
    }
};

```

```

    if (accountAge < 7) {
      return interaction.reply({
        content: 'Your account is too new to verify.',
        ephemeral: true
      });
    }

    const role = interaction.guild.roles.cache.find(r => r.name ===
'Verified');

    if (!role) {
      return interaction.reply({
        content: 'Verified role not found.',
        ephemeral: true
      });
    }

    await member.roles.add(role);

    await interaction.reply({
      content: 'You are now verified.',
      ephemeral: true
    });
  }
};

```

8. Alt Detector

src/events/altDetector.js

```

const {
  EmbedBuilder
} = require('discord.js');

module.exports = {
  name: 'guildMemberAdd',

  async execute(member) {

    const age = Math.floor(
      (Date.now() - member.user.createdTimestamp) /
      (1000 * 60 * 60 * 24)
    );

    if (age > 7) return;
  }
};

```

```

const logChannel = member.guild.channels.cache.find(
  c => c.name === 'security-logs'
);

if (!logChannel) return;

const embed = new EmbedBuilder()
  .setTitle('⚠ Suspicious Account Detected')
  .addFields(
    { name: 'User', value: `${member.user.tag}` },
    { name: 'Account Age', value: `${age} days` }
  )
  .setColor('Red');

logChannel.send({ embeds: [embed] });
}
};

```

9. Anti Nuke Event

src/events/antiChannelDelete.js

```

const {
  AuditLogEvent
} = require('discord.js');

module.exports = {
  name: 'channelDelete',

  async execute(channel) {

    const logs = await channel.guild.fetchAuditLogs({
      type: AuditLogEvent.ChannelDelete,
      limit: 1
    });

    const entry = logs.entries.first();

    if (!entry) return;

    const executor = entry.executor;

    const member = await channel.guild.members.fetch(executor.id);

    if (!member.bannable) return;

    await member.ban({

```

```
        reason: 'Anti Nuke Protection'
    });
}
};
```

10. Event Loader

src/handlers/eventHandler.js

```
const fs = require('fs');

module.exports = (client) => {

    const eventFiles = fs.readdirSync('./src/events')
        .filter(file => file.endsWith('.js'));

    for (const file of eventFiles) {

        const event = require(`../events/${file}`);

        client.on(event.name, (...args) => event.execute(...args));
    }
};
```

11. Main Bot File

src/index.js

```
require('dotenv').config();

const {
    Client,
    GatewayIntentBits
} = require('discord.js');

const loadEvents = require('../handlers/eventHandler');

const client = new Client({
    intents: [
        GatewayIntentBits.Guilds,
        GatewayIntentBits.GuildMembers,
        GatewayIntentBits.GuildMessages,
        GatewayIntentBits.MessageContent
    ]
});
```



```
    ]  
  });  
  
  loadEvents(client);  
  
  client.login(process.env.TOKEN);
```

12. Railway Variables

```
TOKEN=  
DATABASE_URL=  
CLIENT_ID=
```

13. Recommended Channels

Create:

- security-logs
- attendance-logs
- verification

14. Recommended Roles

Create:

- Verified
- Quarantine

15. Recommended Permissions

Bot Needs:

- Administrator
 - Manage Roles
 - Ban Members
 - View Audit Logs
 - Manage Channels
-

16. Deploy To Railway

Push to GitHub:

```
git add .  
git commit -m "Added security systems"  
git push
```

Railway automatically redeploys.